



LOW RISK

GRADE A+ · PROFESSIONAL

# BitGuard Token & Liquidity Locker

LOCK · Solana · Token / liquidity locker

FgTanvmeJJwfw8wbDFf5LMz2Hhrqgz1XAKAZsqDWgeKJ

Our Auditors reviewed the Solana program ID FgTanvmeJJwfw8wbDFf5LMz2Hhrqgz1XAKAZsqDWgeKJ, identified as BitGuard Token & Liquidity Locker (LOCK), and discovered the information below. A Two-Step process was used to analyze the smart contract source code; first using AI-powered security audit tools and then manually. The program is upgradeable; authority DajXsKvDP1AYu4vKwwgct6G8YhPYgNsX19a62ED8aVvJ can replace the bytecode and void current lock semantics. Retrieved Anchor source contains four UncheckedAccount fields, only one of which appears address-pinned, so owner and type checks are incomplete unless further constraints are added. Source shows no admin or emergency path that moves customer locked assets, and extend\_unlock is forward-only, preventing lock shortening in this implementation. Residual risk therefore centers on upgrade-key custody and on account-validation gaps around those unchecked fields. Confirm the live explorer page still matches the published security.txt seal before relying on this deployment.

| CLASS                    | VERIFIED | UPGRADEABLE | CHECKLIST |
|--------------------------|----------|-------------|-----------|
| Token / liquidity locker | Yes      | Yes         | 322       |

## Identity

| FIELD          | VALUE                                        |
|----------------|----------------------------------------------|
| Name / symbol  | BitGuard Token & Liquidity Locker / LOCK     |
| Address        | FgTanvmeJJwfw8wbDFf5LMz2Hhrqgz1XAKAZsqDWgeKJ |
| Network        | Solana                                       |
| Standard       | Token / liquidity locker                     |
| Decimals       | n/a                                          |
| Supply         | n/a                                          |
| Creator        | —                                            |
| Creation tx    | —                                            |
| Compiler       | anchor-lang                                  |
| Optimization   | —                                            |
| Proxy          | BPF upgradeable loader                       |
| Implementation | DLsixG9Ano57RToipNHs514xN1kLgtP2cJtKDev4FaeP |
| Program ID     | FgTanvmeJJwfw8wbDFf5LMz2Hhrqgz1XAKAZsqDWgeKJ |
| Bytecode size  | —                                            |
| Issued         | Fri, 04 Sep 2026 02:26:40 GMT                |

## Control matrix

| STATUS   | CONTROL | DETAIL |
|----------|---------|--------|
| IDENTITY | —       | —      |

| STATUS       | CONTROL                          | DETAIL                                                                                                                                                                                                                                               |
|--------------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| INFO         | Contract class                   | Classified as Token / liquidity locker · known deployment "BitGuard Token & Liquidity Locker".                                                                                                                                                       |
| PASS         | Source verification              | On-chain security.txt is published (Solscan Security.txt: TRUE) · BitGuard Locker · source <a href="https://locker.bbspectrum.com">https://locker.bbspectrum.com</a> . Solscan Program is verified: TRUE (security.txt / source published on-chain). |
| PASS         | Security.txt                     | TRUE · BitGuard Locker · <a href="https://locker.bbspectrum.com">https://locker.bbspectrum.com</a>                                                                                                                                                   |
| WARN         | Upgradeability                   | Upgradeable: Yes. Upgrade authority DajXsKvDP1AYu4vKwwgct6G8YhPYgNsX19a62ED8aVvJ. Last deploy slot 443750835. An upgrade can replace this program.                                                                                                   |
| TOKENOMICS — |                                  |                                                                                                                                                                                                                                                      |
| N/A          | Liquidity                        | Not applicable. This is not a tradable token (locker, program, wallet, or other contract). Supply, mintability, holder concentration, and DEX liquidity apply only to tokens. DEX pool quotes are out of scope.                                      |
| N/A          | Current supply                   | Not applicable. This is not a tradable token (locker, program, wallet, or other contract). Supply, mintability, holder concentration, and DEX liquidity apply only to tokens. totalSupply is a token field.                                          |
| N/A          | Maximum supply                   | Not applicable. This is not a tradable token (locker, program, wallet, or other contract). Supply, mintability, holder concentration, and DEX liquidity apply only to tokens. A max supply cap is a token field.                                     |
| N/A          | Mintability                      | Not applicable. This is not a tradable token (locker, program, wallet, or other contract). Supply, mintability, holder concentration, and DEX liquidity apply only to tokens. Mint authority applies to token mints, not this contract class.        |
| N/A          | Largest wallet                   | Not applicable. This is not a tradable token (locker, program, wallet, or other contract). Supply, mintability, holder concentration, and DEX liquidity apply only to tokens. There is no token holder distribution to rank.                         |
| N/A          | Top-10 concentration             | Not applicable. This is not a tradable token (locker, program, wallet, or other contract). Supply, mintability, holder concentration, and DEX liquidity apply only to tokens. There is no token holder table.                                        |
| N/A          | Owner share                      | Not applicable. This is not a tradable token (locker, program, wallet, or other contract). Supply, mintability, holder concentration, and DEX liquidity apply only to tokens. Owner token-balance share does not apply.                              |
| N/A          | Creator share                    | Not applicable. This is not a tradable token (locker, program, wallet, or other contract). Supply, mintability, holder concentration, and DEX liquidity apply only to tokens. Creator token-balance share does not apply.                            |
| N/A          | Holder count                     | Not applicable. This is not a tradable token (locker, program, wallet, or other contract). Supply, mintability, holder concentration, and DEX liquidity apply only to tokens. This contract is not a circulating token.                              |
| MARKET —     |                                  |                                                                                                                                                                                                                                                      |
| N/A          | Listed on a DEX                  | Not applicable. DEX listings apply to tokens, not this contract class.                                                                                                                                                                               |
| OWNERSHIP —  |                                  |                                                                                                                                                                                                                                                      |
| PASS         | Admin drain / emergency withdraw | BitGuard V1 documents no emergencyWithdraw on customer assets.                                                                                                                                                                                       |
| PASS         | Unlock can be shortened          | BitGuard V1 is forward-only: newUnlockTime must be "e current"                                                                                                                                                                                       |

## Smart Contract Security Vulnerability Checklist

Applicable checks for this contract class. Token-only items (liquidity, holders, taxes) are omitted for non-token contracts.

| STATUS                   | CHECK                      | RESULT                                                                                                                         |
|--------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| S1. ACCOUNT VALIDATION — |                            |                                                                                                                                |
| PASS                     | Missing account validation | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |

| STATUS                          | CHECK                                         | RESULT                                                                                                                                                                                 |
|---------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PASS                            | Missing owner validation                      | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Missing signer validation                     | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                            | Missing writable-account validation           | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Incorrect account ownership                   | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Account substitution                          | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Account confusion                             | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Account type confusion                        | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Type cosplay                                  | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                            | Incorrect account relationship                | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Missing has_one validation                    | Anchor Accounts structs and #[account] constraints are in retrieved Rust. No matching issue for this check.                                                                            |
| PASS                            | Missing address validation                    | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| WARN                            | Unchecked AccountInfo                         | UncheckedAccount is present in retrieved Rust. Anchor will not check owner/type unless address/owner constraints are added.                                                            |
| WARN                            | Unchecked UncheckedAccount                    | UncheckedAccount is present. Confirm every CHECK field has address/owner/signer constraints.                                                                                           |
| PASS                            | Duplicate mutable accounts                    | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Duplicate account references                  | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Missing account existence validation          | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Incorrect executable-account validation       | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Incorrect system-program validation           | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Incorrect token-program validation            | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                            | Incorrect associated-token-program validation | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| S2. SIGNER / AUTHORITY SECURITY | —                                             | —                                                                                                                                                                                      |
| PASS                            | Missing signer checks                         | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                            | Fake authority                                | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                            | Authority impersonation                       | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |

| STATUS                             | CHECK                                 | RESULT                                                                                                                         |
|------------------------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| PASS                               | Incorrect signer validation           | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| PASS                               | Signer substitution                   | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| PASS                               | Unauthorized signer                   | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| PASS                               | Missing authority relationship checks | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| PASS                               | Authority takeover                    | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| PASS                               | Authority replacement                 | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| PASS                               | Unauthorized authority transfer       | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| PASS                               | Missing multisignature validation     | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                               | Insufficient signer threshold         | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| PASS                               | Privilege escalation                  | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| S3. PDA SECURITY                   | —                                     | —                                                                                                                              |
| PASS                               | PDA seed collision                    | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | Incorrect PDA seeds                   | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | Missing PDA validation                | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | Incorrect PDA derivation              | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | Incorrect program ID                  | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                               | Missing bump validation               | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | Non-canonical bump                    | PDA bumps are stored and reused (canonical bump), not taken as a raw user seed in retrieved Rust.                              |
| PASS                               | User-controlled bump                  | PDA bumps are stored and reused (canonical bump), not taken as a raw user seed in retrieved Rust.                              |
| PASS                               | PDA substitution                      | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | PDA account confusion                 | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | PDA authority confusion               | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | PDA seed manipulation                 | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | Cross-instance PDA collision          | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | Missing seeds constraint              | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | Incorrect seeds::program              | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| PASS                               | PDA privilege misuse                  | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |
| S4. CROSS-PROGRAM INVOCATION (CPI) | —                                     | —                                                                                                                              |
| PASS                               | Arbitrary CPI                         | CPI targets the SPL token interface / Anchor SPL, not an arbitrary program id from the caller.                                 |



| STATUS                     | CHECK                                  | RESULT                                                                                                                                                                                                                  |
|----------------------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PASS                       | Unverified CPI target                  | CPI targets the SPL token interface / Anchor SPL, not an arbitrary program id from the caller.                                                                                                                          |
| PASS                       | Malicious CPI program                  | CPI targets the SPL token interface / Anchor SPL, not an arbitrary program id from the caller.                                                                                                                          |
| PASS                       | Incorrect program ID                   | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| INFO                       | CPI account substitution               | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| INFO                       | CPI signer privilege abuse             | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| INFO                       | CPI writable privilege abuse           | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| INFO                       | Unauthorized invoke                    | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| INFO                       | Unauthorized invoke_signed             | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| PASS                       | Incorrect PDA signer seeds             | PDA seeds and bump constraints are declared in retrieved Rust.                                                                                                                                                          |
| INFO                       | CPI privilege escalation               | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| INFO                       | CPI recursion issues                   | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| INFO                       | Malicious callback                     | Solana is not EVM reentrancy. Direct self-recursion is possible; indirect reentrancy through another program is restricted by the runtime. Retrieved source still needs CPI-after-state-update review where CPI exists. |
| INFO                       | CPI-induced state manipulation         | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| PASS                       | Untrusted external program interaction | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| INFO                       | Missing CPI account validation         | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| S5. ACCOUNT INITIALIZATION | —                                      | —                                                                                                                                                                                                                       |
| PASS                       | Reinitialization attack                | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                                                        |
| PASS                       | Missing initialization check           | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                                                        |
| PASS                       | Double initialization                  | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                                                        |
| PASS                       | Initialization takeover                | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                                                        |
| PASS                       | Uninitialized account usage            | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                                                        |
| PASS                       | Incorrect initialization authority     | Signer<info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                                                             |
| PASS                       | Initialization parameter manipulation  | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                                                        |
| PASS                       | Initialization front-running           | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                                                        |
| PASS                       | Account creation DoS                   | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                                                                                                         |
| PASS                       | Incorrect account size                 | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| INFO                       | Incorrect rent handling                | Native SOL movement uses system_program in retrieved Rust. Confirm the treasury/recipient is pinned (address = config.fee_treasury or equivalent).                                                                      |

| STATUS                               | CHECK                                | RESULT                                                                                                                                                                                 |
|--------------------------------------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>S6. ACCOUNT CLOSING / REVIVAL</b> |                                      |                                                                                                                                                                                        |
| PASS                                 | Account revival                      | No account-close constraint in retrieved Rust.                                                                                                                                         |
| PASS                                 | Closed-account reuse                 | No account-close constraint in retrieved Rust.                                                                                                                                         |
| PASS                                 | Improper account closure             | No account-close constraint in retrieved Rust.                                                                                                                                         |
| PASS                                 | Unauthorized account closure         | No account-close constraint in retrieved Rust.                                                                                                                                         |
| PASS                                 | Incorrect close authority            | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                                 | State persistence after closure      | No account-close constraint in retrieved Rust.                                                                                                                                         |
| INFO                                 | Lamport manipulation                 | Native SOL movement uses system_program in retrieved Rust. Confirm the treasury/recipient is pinned (address = config.fee_treasury or equivalent).                                     |
| PASS                                 | Close-account race conditions        | No account-close constraint in retrieved Rust.                                                                                                                                         |
| PASS                                 | Reinitialization after closure       | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                       |
| <b>S7. REMAINING ACCOUNTS</b>        |                                      |                                                                                                                                                                                        |
| PASS                                 | Unvalidated remaining accounts       | No remaining_accounts iteration in retrieved Rust.                                                                                                                                     |
| PASS                                 | Arbitrary remaining accounts         | No remaining_accounts iteration in retrieved Rust.                                                                                                                                     |
| PASS                                 | Incorrect remaining-account count    | No remaining_accounts iteration in retrieved Rust.                                                                                                                                     |
| PASS                                 | Account substitution                 | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                 | Duplicate remaining accounts         | No remaining_accounts iteration in retrieved Rust.                                                                                                                                     |
| PASS                                 | Unverified ownership                 | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                 | Unverified PDA                       | PDA seeds and bump constraints are declared in retrieved Rust.                                                                                                                         |
| PASS                                 | Unverified token account             | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                 | Unverified mint                      | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                                 | Unverified authority                 | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                                 | Malicious remaining-account ordering | No remaining_accounts iteration in retrieved Rust.                                                                                                                                     |
| INFO                                 | Remaining-account CPI abuse          | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                |
| <b>S8. ANCHOR-SPECIFIC</b>           |                                      |                                                                                                                                                                                        |
| PASS                                 | Incorrect #[account] constraints     | Anchor Accounts structs and #[account] constraints are in retrieved Rust. No matching issue for this check.                                                                            |
| PASS                                 | Missing Signer                       | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                                 | Incorrect Account                    | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| WARN                                 | Unsafe UncheckedAccount              | UncheckedAccount is present. Confirm every CHECK field has address/owner/signer constraints.                                                                                           |

| STATUS                                 | CHECK                              | RESULT                                                                                                                                                                                 |
|----------------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PASS                                   | Missing has_one                    | Anchor Accounts structs and #[account] constraints are in retrieved Rust. No matching issue for this check.                                                                            |
| PASS                                   | Missing constraint                 | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                   | Incorrect seeds                    | PDA seeds and bump constraints are declared in retrieved Rust.                                                                                                                         |
| PASS                                   | Incorrect bump                     | PDA seeds and bump constraints are declared in retrieved Rust.                                                                                                                         |
| PASS                                   | Incorrect init                     | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                   | Incorrect init_if_needed           | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                   | Unsafe realloc                     | Anchor Accounts structs and #[account] constraints are in retrieved Rust. No matching issue for this check.                                                                            |
| PASS                                   | Incorrect close                    | No account-close constraint in retrieved Rust.                                                                                                                                         |
| PASS                                   | Account reload issues              | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                   | Account discriminator bypass       | Anchor Accounts structs and #[account] constraints are in retrieved Rust. No matching issue for this check.                                                                            |
| PASS                                   | Manual deserialization flaws       | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                   | Type cosplay                       | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                                   | Constraint bypass                  | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                   | Anchor version vulnerabilities     | Anchor Accounts structs and #[account] constraints are in retrieved Rust. No matching issue for this check.                                                                            |
| PASS                                   | Incorrect account relationships    | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| S9. SPL<br>TOKEN /<br>TOKEN<br>ACCOUNT | —                                  | —                                                                                                                                                                                      |
| PASS                                   | Incorrect mint validation          | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                                   | Incorrect token-account validation | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                                   | Incorrect token authority          | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                                   | Token-account substitution         | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                                   | Mint substitution                  | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                                   | Freeze-authority abuse             | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                                   | Mint-authority abuse               | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |

| STATUS                            | CHECK                          | RESULT                                                                                                                                                                                 |
|-----------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PASS                              | Unauthorized minting           | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                              | Unauthorized burning           | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                              | Unauthorized transfer          | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                              | Unauthorized freeze            | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                              | Unauthorized thaw              | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                              | Token decimal mismatch         | Retrieved Rust uses require! / checked paths rather than unwrap on arithmetic sites scanned.                                                                                           |
| PASS                              | Token amount manipulation      | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                              | Token-account ownership errors | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                              | Token-program substitution     | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| N/A                               | Token-2022 incompatibility     | Not applicable. Token-2022 mint extensions apply to SPL mints, not this program class.                                                                                                 |
| N/A                               | Transfer-hook exploitation     | Not applicable. Token-2022 mint extensions apply to SPL mints, not this program class.                                                                                                 |
| PASS                              | Transfer-fee manipulation      | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| N/A                               | Permanent-delegate abuse       | Not applicable. Token-2022 mint extensions apply to SPL mints, not this program class.                                                                                                 |
| PASS                              | Token extension misuse         | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| S10.<br>ARITHMETIC /<br>NUMERICAL | —                              | —                                                                                                                                                                                      |
| PASS                              | Integer overflow               | Retrieved Rust uses require! / checked paths rather than unwrap on arithmetic sites scanned.                                                                                           |
| PASS                              | Integer underflow              | Retrieved Rust uses require! / checked paths rather than unwrap on arithmetic sites scanned.                                                                                           |
| PASS                              | Division by zero               | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                              | Integer truncation             | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                              | Precision loss                 | Retrieved Rust uses require! / checked paths rather than unwrap on arithmetic sites scanned.                                                                                           |
| PASS                              | Rounding errors                | Retrieved Rust uses require! / checked paths rather than unwrap on arithmetic sites scanned.                                                                                           |

| STATUS                                             | CHECK                               | RESULT                                                                                                                         |
|----------------------------------------------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| PASS                                               | Decimal mismatch                    | Retrieved Rust uses require! / checked paths rather than unwrap on arithmetic sites scanned.                                   |
| PASS                                               | Unsafe casting                      | Retrieved Rust uses require! / checked paths rather than unwrap on arithmetic sites scanned.                                   |
| PASS                                               | Casting truncation                  | Retrieved Rust uses require! / checked paths rather than unwrap on arithmetic sites scanned.                                   |
| PASS                                               | Signed/unsigned conversion errors   | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Incorrect percentage calculations   | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Incorrect fee calculations          | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Incorrect reward calculations       | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Incorrect token amount calculations | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Incorrect share calculations        | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| N/A                                                | Incorrect price calculations        | Not applicable. Price oracles are out of scope for this contract class.                                                        |
| S11. SOLANA<br>NATIVE<br>PROGRAM<br>SECURITY       | —                                   | —                                                                                                                              |
| PASS                                               | Unsafe AccountInfo handling         | No unwrap/expect/unsafe in retrieved Rust for this check.                                                                      |
| PASS                                               | Missing owner checks                | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Missing signer checks               | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                   |
| PASS                                               | Missing executable checks           | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Incorrect account data parsing      | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Unsafe serialization                | No unwrap/expect/unsafe in retrieved Rust for this check.                                                                      |
| PASS                                               | Unsafe deserialization              | No unwrap/expect/unsafe in retrieved Rust for this check.                                                                      |
| PASS                                               | Borsh parsing issues                | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Account-data corruption             | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Incorrect account-size assumptions  | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Manual discriminator errors         | Anchor Accounts structs and #[account] constraints are in retrieved Rust. No matching issue for this check.                    |
| PASS                                               | Incorrect program-ID validation     | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                               | Incorrect sysvar validation         | Sysvar access uses Clock::get / Rent::get (runtime sysvar), not a caller-passed fake account, in retrieved Rust.               |
| S12. SYSVAR<br>/ INSTRUCTIO<br>N INTROSPE<br>CTION | —                                   | —                                                                                                                              |

| STATUS                                | CHECK                                  | RESULT                                                                                                                                                                                                                  |
|---------------------------------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PASS                                  | Sysvar spoofing                        | Sysvar access uses Clock::get / Rent::get (runtime sysvar), not a caller-passed fake account, in retrieved Rust.                                                                                                        |
| PASS                                  | Incorrect sysvar validation            | Sysvar access uses Clock::get / Rent::get (runtime sysvar), not a caller-passed fake account, in retrieved Rust.                                                                                                        |
| PASS                                  | Instructions sysvar manipulation       | Sysvar access uses Clock::get / Rent::get (runtime sysvar), not a caller-passed fake account, in retrieved Rust.                                                                                                        |
| PASS                                  | Transaction introspection flaws        | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| PASS                                  | Missing instruction-index validation   | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| PASS                                  | Ed25519 verification bypass            | Sysvar access uses Clock::get / Rent::get (runtime sysvar), not a caller-passed fake account, in retrieved Rust.                                                                                                        |
| PASS                                  | Secp256k1 verification bypass          | Sysvar access uses Clock::get / Rent::get (runtime sysvar), not a caller-passed fake account, in retrieved Rust.                                                                                                        |
| PASS                                  | Signature verification logic errors    | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| PASS                                  | Cross-instruction authorization bypass | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| PASS                                  | Missing transaction-level constraints  | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| S13. REENTRANCY / RECURSIVE EXECUTION |                                        |                                                                                                                                                                                                                         |
| INFO                                  | Reentrancy                             | Solana is not EVM reentrancy. Direct self-recursion is possible; indirect reentrancy through another program is restricted by the runtime. Retrieved source still needs CPI-after-state-update review where CPI exists. |
| INFO                                  | CPI recursion                          | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| PASS                                  | Cross-program recursive execution      | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| INFO                                  | State inconsistency during CPI         | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                                                 |
| INFO                                  | Callback exploitation                  | Solana is not EVM reentrancy. Direct self-recursion is possible; indirect reentrancy through another program is restricted by the runtime. Retrieved source still needs CPI-after-state-update review where CPI exists. |
| PASS                                  | State-update-after-CPI vulnerabilities | PDA seeds and bump constraints are declared in retrieved Rust.                                                                                                                                                          |
| PASS                                  | Read-before-write inconsistencies      | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                                                          |
| S14. SOL / LAMPORT HANDLING           |                                        |                                                                                                                                                                                                                         |
| INFO                                  | Unauthorized SOL withdrawal            | Native SOL movement uses system_program in retrieved Rust. Confirm the treasury/recipient is pinned (address = config.fee_treasury or equivalent).                                                                      |
| INFO                                  | Lamport manipulation                   | Native SOL movement uses system_program in retrieved Rust. Confirm the treasury/recipient is pinned (address = config.fee_treasury or equivalent).                                                                      |
| INFO                                  | Incorrect lamport accounting           | Native SOL movement uses system_program in retrieved Rust. Confirm the treasury/recipient is pinned (address = config.fee_treasury or equivalent).                                                                      |
| INFO                                  | Forced SOL transfers                   | Native SOL movement uses system_program in retrieved Rust. Confirm the treasury/recipient is pinned (address = config.fee_treasury or equivalent).                                                                      |
| INFO                                  | Incorrect rent handling                | Native SOL movement uses system_program in retrieved Rust. Confirm the treasury/recipient is pinned (address = config.fee_treasury or equivalent).                                                                      |



| STATUS                                  | CHECK                              | RESULT                                                                                                                         |
|-----------------------------------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| PASS                                    | Rent-exemption failures            | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                    | Unauthorized account funding       | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                    | Balance manipulation               | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                    | Incorrect SOL transfer authority   | Signer<'info'> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                  |
| PASS                                    | System-program substitution        | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| S15. TRANSACTION / INSTRUCTION SECURITY | —                                  | —                                                                                                                              |
| PASS                                    | Transaction-order dependence       | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| INFO                                    | Front-running                      | Solana leaders can order transactions in a slot. Sandwich/TOD is market structure, not a missing require() in this program.    |
| PASS                                    | Back-running                       | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| INFO                                    | Sandwich attacks                   | Solana leaders can order transactions in a slot. Sandwich/TOD is market structure, not a missing require() in this program.    |
| INFO                                    | Transaction replay                 | Solana leaders can order transactions in a slot. Sandwich/TOD is market structure, not a missing require() in this program.    |
| INFO                                    | Instruction replay                 | Solana leaders can order transactions in a slot. Sandwich/TOD is market structure, not a missing require() in this program.    |
| PASS                                    | Duplicate instruction execution    | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                    | Instruction-order manipulation     | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| INFO                                    | Missing nonce protection           | Solana leaders can order transactions in a slot. Sandwich/TOD is market structure, not a missing require() in this program.    |
| PASS                                    | Missing expiration                 | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                                    | Stale transaction execution        | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| INFO                                    | Blockhash expiration handling      | Solana leaders can order transactions in a slot. Sandwich/TOD is market structure, not a missing require() in this program.    |
| PASS                                    | Transaction simulation assumptions | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| S16. TOKEN LOCKER                       | —                                  | —                                                                                                                              |
| PASS                                    | Unauthorized unlock                | No matching issue for this locker check in retrieved Rust.                                                                     |
| PASS                                    | Early unlock                       | extend_unlock requires new_unlock > current unlock_ts (forward-only) in retrieved Rust.                                        |
| PASS                                    | Lock bypass                        | extend_unlock requires new_unlock > current unlock_ts (forward-only) in retrieved Rust.                                        |
| PASS                                    | Lock-time manipulation             | extend_unlock requires new_unlock > current unlock_ts (forward-only) in retrieved Rust.                                        |
| PASS                                    | Unlock timestamp manipulation      | extend_unlock requires new_unlock > current unlock_ts (forward-only) in retrieved Rust.                                        |



| STATUS                      | CHECK                             | RESULT                                                                                                                                                                                 |
|-----------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PASS                        | Lock amount manipulation          | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Locked-balance manipulation       | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Lock-account substitution         | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | PDA lock-account substitution     | PDA seeds and bump constraints are declared in retrieved Rust.                                                                                                                         |
| PASS                        | Unauthorized withdrawal           | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Unauthorized token recovery       | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Unauthorized emergency withdrawal | No emergencyWithdraw / admin drain of customer assets in retrieved Rust.                                                                                                               |
| PASS                        | Duplicate withdrawal              | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Double claim                      | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Token-account substitution        | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                        | Mint substitution                 | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                        | Lock ownership takeover           | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Lock authority takeover           | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                        | Reinitialization                  | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                       |
| INFO                        | CPI abuse                         | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                |
| N/A                         | Token-2022 compatibility issues   | Not applicable. Token-2022 mint extensions apply to SPL mints, not this program class.                                                                                                 |
| S17.<br>LIQUIDITY<br>LOCKER | —                                 | —                                                                                                                                                                                      |
| PASS                        | Unauthorized LP withdrawal        | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Early LP unlock                   | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | LP ownership takeover             | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| N/A                         | LP-token substitution             | Not applicable. Pool/liquidity/share-price attacks apply to tokens and AMMs, not this program class.                                                                                   |
| PASS                        | LP amount manipulation            | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| N/A                         | Pool substitution                 | Not applicable. Pool/liquidity/share-price attacks apply to tokens and AMMs, not this program class.                                                                                   |
| PASS                        | Pair/account substitution         | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| N/A                         | Liquidity manipulation            | Not applicable. Pool/liquidity/share-price attacks apply to tokens and AMMs, not this program class.                                                                                   |
| PASS                        | Lock bypass                       | extend_unlock requires new_unlock > current unlock_ts (forward-only) in retrieved Rust.                                                                                                |
| PASS                        | Unlock-time manipulation          | extend_unlock requires new_unlock > current unlock_ts (forward-only) in retrieved Rust.                                                                                                |
| PASS                        | Duplicate LP withdrawal           | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Double claim                      | No matching issue for this locker check in retrieved Rust.                                                                                                                             |
| PASS                        | Unauthorized recovery             | No matching issue for this locker check in retrieved Rust.                                                                                                                             |

| STATUS                           | CHECK                             | RESULT                                                                                                                                  |
|----------------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| PASS                             | Unauthorized emergency withdrawal | No emergencyWithdraw / admin drain of customer assets in retrieved Rust.                                                                |
| PASS                             | PDA manipulation                  | PDA seeds and bump constraints are declared in retrieved Rust.                                                                          |
| INFO                             | CPI abuse                         | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program. |
| PASS                             | Token-program substitution        | No matching issue for this locker check in retrieved Rust.                                                                              |
| N/A                              | Token-2022 compatibility issues   | Not applicable. Token-2022 mint extensions apply to SPL mints, not this program class.                                                  |
| S18. UPGRADE / PROGRAM AUTHORITY |                                   |                                                                                                                                         |
| PASS                             | Initialization after upgrade      | Accounts use Anchor init (not init if needed) in retrieved Rust.                                                                        |
| PASS                             | Upgrade-authority takeover        | Signer<info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                             |
| PASS                             | Unprotected upgrade authority     | Signer<info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                             |
| PASS                             | Upgrade authority not revoked     | Signer<info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                             |
| INFO                             | Excessive CPI                     | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program. |
| PASS                             | Excessive account allocation      | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.          |
| PASS                             | Account creation DoS              | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                         |
| INFO                             | CPI abuse                         | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program. |
| PASS                             | Token-program substitution        | No matching issue for this locker check in retrieved Rust.                                                                              |
| PASS                             | Reallocation DoS                  | Anchor Accounts structs and #[account] constraints are in retrieved Rust. No matching issue for this check.                             |
| N/A                              | Token-2022 compatibility issues   | Not applicable. Token-2022 mint extensions apply to SPL mints, not this program class.                                                  |
| PASS                             | Initialization after upgrade      | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                        |
| PASS                             | Compute-unit exhaustion           | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                         |
| PASS                             | PDA manipulation                  | PDA seeds and bump constraints are declared in retrieved Rust.                                                                          |
| S19. DOS / RESOURCE EXHAUSTION   |                                   |                                                                                                                                         |
| PASS                             | Account creation DoS              | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                         |
| PASS                             | Compute-unit exhaustion           | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                         |
| INFO                             | Excessive CPI                     | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program. |
| PASS                             | Excessive account loading         | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.          |
| PASS                             | Excessive account allocation      | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.          |
| PASS                             | Reallocation DoS                  | Anchor Accounts structs and #[account] constraints are in retrieved Rust. No matching issue for this check.                             |

| STATUS                          | CHECK                           | RESULT                                                                                                                         |
|---------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| PASS                            | Transaction-size exhaustion     | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                |
| PASS                            | Stack exhaustion                | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                |
| PASS                            | Heap exhaustion                 | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                |
| PASS                            | Loop-based DoS                  | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                |
| PASS                            | Unbounded iteration             | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                |
| PASS                            | Malicious remaining accounts    | No remaining_accounts iteration in retrieved Rust.                                                                             |
| PASS                            | Rent-related DoS                | No remaining_accounts loop or unbounded loop in retrieved Rust.                                                                |
| PASS                            | Account-lock contention         | No matching issue for this locker check in retrieved Rust.                                                                     |
| S20. BUSINESS LOGIC / ECONOMIC  | —                               | —                                                                                                                              |
| PASS                            | Incorrect state transitions     | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Broken invariants               | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Incorrect accounting            | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Unauthorized state modification | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Missing state validation        | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | State desynchronization         | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Incorrect fee logic             | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Incorrect reward logic          | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Incorrect ownership logic       | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Incorrect lock logic            | No matching issue for this locker check in retrieved Rust.                                                                     |
| PASS                            | Incorrect withdrawal logic      | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| N/A                             | Incorrect liquidity accounting  | Not applicable. Pool/liquidity/share-price attacks apply to tokens and AMMs, not this program class.                           |
| PASS                            | Economic exploits               | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Incentive manipulation          | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Centralization risks            | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Admin abuse                     | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS                            | Emergency-function abuse        | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| S21. DEPLOYMENT / CONFIGURATION | —                               | —                                                                                                                              |

| STATUS                                     | CHECK                                  | RESULT                                                                                                                                                                                 |
|--------------------------------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PASS                                       | Incorrect program ID                   | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                       | Incorrect PDA seeds                    | PDA seeds and bump constraints are declared in retrieved Rust.                                                                                                                         |
| PASS                                       | Incorrect deployment configuration     | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                       | Wrong upgrade authority                | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                                       | Wrong token mint                       | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                                       | Wrong token program                    | Retrieved Rust types mint/token accounts as InterfaceAccount (Mint/TokenAccount), not raw AccountInfo — substitution still needs the token-program pin, which TokenInterface provides. |
| PASS                                       | Wrong treasury address                 | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| N/A                                        | Wrong oracle address                   | Not applicable. Price oracles are out of scope for this contract class.                                                                                                                |
| PASS                                       | Wrong DEX/program address              | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                       | Incorrect production configuration     | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                       | Debug/test configuration in production | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                       | Uninitialized production accounts      | Accounts use Anchor init (not init_if_needed) in retrieved Rust.                                                                                                                       |
| PASS                                       | Missing authority revocation           | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| S22. SOLANA<br>-SPECIFIC<br>CODE<br>REVIEW | —                                      | —                                                                                                                                                                                      |
| PASS                                       | Unsafe unsafe Rust                     | No unwrap/expect/unsafe in retrieved Rust for this check.                                                                                                                              |
| PASS                                       | Panic conditions                       | No unwrap/expect/unsafe in retrieved Rust for this check.                                                                                                                              |
| PASS                                       | unwrap() abuse                         | No unwrap/expect/unsafe in retrieved Rust for this check.                                                                                                                              |
| PASS                                       | expect() abuse                         | No unwrap/expect/unsafe in retrieved Rust for this check.                                                                                                                              |
| PASS                                       | Unchecked arithmetic                   | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                       | Unchecked casts                        | Retrieved Rust uses require! / checked paths rather than unwrap on arithmetic sites scanned.                                                                                           |
| PASS                                       | Manual account parsing                 | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                       | Manual PDA derivation                  | PDA seeds and bump constraints are declared in retrieved Rust.                                                                                                                         |
| PASS                                       | Manual signer verification             | Signer<'info> / has_one constraints are in retrieved Rust. Instruction handlers require a signing authority.                                                                           |
| PASS                                       | Manual owner verification              | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| PASS                                       | Manual token validation                | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source.                                                         |
| INFO                                       | Unsafe CPI construction                | CPI (token transfer / invoke) is present. Retrieved Rust pins the token program via TokenInterface rather than a caller-chosen program.                                                |

| STATUS | CHECK                       | RESULT                                                                                                                         |
|--------|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| PASS   | Hard-coded addresses        | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS   | Hard-coded authorities      | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS   | Missing error handling      | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS   | Incomplete error handling   | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS   | Incorrect error propagation | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS   | Unchecked return values     | Retrieved Rust uses Anchor account constraints (Signer / #[account] / seeds). No matching issue for this check in that source. |
| PASS   | Inconsistent state updates  | PDA seeds and bump constraints are declared in retrieved Rust.                                                                 |

## Findings

| SEV    | TITLE                                     | DETAIL                                                                                                                                                                                                                                                                                                |
|--------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MEDIUM | UncheckedAccount in Anchor accounts       | 4 UncheckedAccount field(s) in retrieved Rust. Anchor skips owner/type checks unless you add address/signer/owner constraints. 1 appear address-pinned. · Remediation: Replace UncheckedAccount with Account/SystemAccount, or pin address + owner on every remaining CHECK field.                    |
| INFO   | No admin drain of locked assets in source | Retrieved Rust has no emergencyWithdraw / admin path that moves customer tokens. Residual risk is upgrade authority replacing this bytecode. · Remediation: Keep upgrade authority on a hardware wallet / multisig, or revoke it.                                                                     |
| INFO   | Unlock time is forward-only in source     | extend_unlock requires new_unlock > current_unlock_ts. Owners cannot shorten a lock in this implementation. · Remediation: Keep this check in every future upgrade.                                                                                                                                   |
| INFO   | Known BitGuard Token & Liquidity Locker   | This address matches a documented BitGuard V1 locker core (immutable, two-step ownership, forward-only extend, no admin drain of customer locks). Still verify the live explorer page against this seal. · Remediation: Keep lock records (ID, vault, owner wallet, unlock UTC) with the client file. |

## What these risks mean on Solana

OWASP's Top 10 is written for Ethereum-style contracts. Each row is the same risk, rewritten for this Solana mint.

| STATUS | ID   | WHAT THIS MEANS                                                                                                                                   | THIS TOKEN                                                                                    |
|--------|------|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| INFO   | SC01 | On Solana, a BPF upgradeable program can be replaced by its upgrade authority. This is not a token mint, so mint/freeze authorities do not apply. | See the Upgradeability row. Mint-authority language is not applicable to a program or locker. |
| INFO   | SC02 | This class is about oracle/DEX price manipulation. It applies to tokens used as collateral, not to locker or bot programs.                        | Not applicable to this contract class.                                                        |
| INFO   | SC03 | This address is a program. Review upgrade authority, CPI, and published source (security.txt).                                                    | Not an SPL mint. Token-2022 mint extras do not apply.                                         |
| INFO   | SC04 | Transfer hooks apply to Token-2022 mints, not to this program.                                                                                    | Not applicable.                                                                               |
| INFO   | SC05 | Solana programs are not EVM contracts. Reentrancy is a different (CPI) question.                                                                  | Reviewed as a Solana program, not an EVM token.                                               |
| INFO   | SC06 | CPI to other programs is the Solana equivalent. Review the program IDL / source.                                                                  | Not applicable as an ERC-20 external-call scan.                                               |
| INFO   | SC07 | Flash-loan risk applies to AMM/lending programs, not automatically to a locker.                                                                   | Not applicable unless this program borrows in one transaction.                                |



| STATUS | ID   | WHAT THIS MEANS                                                                  | THIS TOKEN                                                             |
|--------|------|----------------------------------------------------------------------------------|------------------------------------------------------------------------|
| INFO   | SC08 | Rust/Solana programs do not use Solidity 0.8 overflow rules.                     | Not applicable as an EVM arithmetic check.                             |
| INFO   | SC09 | Slot hashes can be biased. Only relevant if this program uses them for outcomes. | Not applicable unless the program implements a lottery or random mint. |
| INFO   | SC10 | Compute-budget and account-lock DoS are Solana-specific.                         | Not scored as an EVM gas-limit loop.                                   |

## Summary

FgTanvmeJJwfw8wbDFf5LMz2Hhrqgz1XAKAZsqDWgeKJ is a verified Solana program (security.txt / explorer), presented as BitGuard Token & Liquidity Locker under the BPF upgradeable loader. Implementation account DLsixG9Ano57RToipNH514xN1kLgtP2cJtKDeV4FaeP holds the current bytecode. The address aligns with a documented BitGuard V1 locker core: SOL-denominated service fees, no admin drain of customer locks, forward-only unlock extension, and config changes for fee and treasury gated by a short timelock.

The program is upgradeable. Upgrade authority is DajXsKvDP1AYu4vKwwgct6G8YhPYgNsX19a62ED8aVvJ (last deploy slot 443750835). Source comments describe immutable lock rules, but the loader allows full replacement; those rules bind only while this implementation remains live.

Config authority may propose fee and treasury updates subject to the five-minute delay; retrieved Rust does not give that path custody of locked customer tokens. The material residual risks are compromise or misuse of the upgrade authority and incomplete Anchor checks on UncheckedAccount fields. Any subsequent upgrade should be treated as a reset of trust assumptions for existing locks.

## Recommendations

| # | ACTION                                                                                                                                 |
|---|----------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Place the upgrade authority under multisig or timelocked governance, or renounce it, so lock bytecode cannot be replaced unilaterally. |
| 2 | Replace UncheckedAccount fields with typed Account or InterfaceAccount types and explicit address, owner, and signer constraints.      |
| 3 | Address-pin every remaining unchecked account to the expected program, mint, or PDA seeds in the accounts struct.                      |
| 4 | Keep security.txt contacts, policy URL, and source linkage current and consistent with the explorer verification page.                 |
| 5 | Publish the upgrade-authority holder, last deploy slot, and any planned path to immutability for lock counterparties.                  |
| 6 | Preserve no-admin-drain and forward-only extend_unlock invariants in any future upgrade; regression-test both before deployment.       |
| 7 | Verify fee and treasury apply instructions still enforce the effective timestamps and cannot bypass the timelock.                      |
| 8 | Re-verify the on-chain program hash against the sealed source after every upgrade before accepting new locks.                          |

## Scope & methodology

| SCOPE                                                            |
|------------------------------------------------------------------|
| Contract / program identity and deployment provenance            |
| Ownership, roles, mint / freeze / pause / upgrade authorities    |
| Tokenomics, supply, holders, liquidity concentration             |
| Malicious-token behavior (honeypot, tax, blacklist, traps)       |
| EVM vulnerability classes aligned to OWASP Smart Contract Top 10 |
| Solana SPL / Token-2022 extensions and custom program risk       |



## METHODOLOGY

On-chain RPC reads of bytecode, storage slots (EIP-1967), ERC-20 / SPL metadata, and Token-2022 TLV extensions.

Verified-source retrieval (Sourcify / Blockscout) followed by a dedicated static-analysis engine — not an LLM — covering OWASP Smart Contract Top 10 classes and token-trap patterns.

Selector extraction from deployed bytecode to detect privileged entrypoints even when source is unverified.

Independent market and liquidity quotes from multiple public feeds; missing quotes are reported as coverage gaps, not zero liquidity.

Security-signal aggregation (honeypot simulation, mint/freeze authorities, holder concentration) with vendor names withheld from the sealed letter.

BitGuard synthesis of the already-computed evidence into an engagement letter. The model does not invent on-chain facts.

## Disclaimer

*BitGuard Professional Smart Contract Assessment. Automated static analysis, on-chain reads, authority mapping, and multi-model review of publicly available data. This engagement snapshot is not a substitute for a human formal-verification review, legal opinion, or insurance underwriting. Privileged keys, upgrades, and off-chain operators can change after this report is sealed.*